(

inicio( )

instrucciones( ) ( )

functions func main( )

) (block {)

( ) instrucciones( ) (}

instrucciones( )

varDcl mut puntos( ) ( ) ( )

type int assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "=== Archivo de prueba de funcionalidades intermedias ==="

instrucciones( )

imprimir println( )

( )

expr "==== Manejo de entornos ===="

instrucciones( )

varDcl mut puntosEntornos( ) ( ) ( )

type int assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "Variable redeclarada en el mismo entorno"

instrucciones( )

varDcl mut a( ) ( ) ( )

type int assign = expr 10

instrucciones( )

imprimir println( )

( ) ,( )

expr "a =" expr a

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr a expr 10

block { }

( ) ( )

instrucciones( )

varAsign puntosEntornos =( )

expr( ) +( )

expr puntosEntornos expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Detección de redeclaración en mismo entorno: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Detección de redeclaración en mismo entorno: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nVariable redeclarada en un entorno diferente"

instrucciones( )

varDcl mut b( ) ( ) ( )

type int assign = expr 10

instrucciones( )

sIf if( ) ( )

expr true block { }

( ) ( ) ( )

instrucciones( )

varAsign b =( )

expr 20

instrucciones( )

imprimir println( )

( ) ,( )

expr "b dentro del if =" expr b

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr b expr 20

block { }

( ) ( )

instrucciones( )

varAsign puntosEntornos =( )

expr( ) +( )

expr puntosEntornos expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Redeclaración en entorno diferente: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Redeclaración en entorno diferente: incorrecto"

instrucciones( )

imprimir println( )

( ) ,( )

expr "b fuera del if =" expr b

instrucciones( )

imprimir println( )

( )

expr "\nUso de variable en un entorno superior"

instrucciones( )

varDcl mut c( ) ( ) ( )

type int assign = expr 10

instrucciones( )

varDcl mut d( ) ( ) ( )

type int assign = expr 10

instrucciones( )

sIf if( ) ( )

expr true block { }

( ) ( ) ( )

instrucciones( )

varAsign d =( )

expr 20

instrucciones( )

varAsign c =( )

expr 50

instrucciones( )

varAsign d =( )

expr 30

instrucciones( )

imprimir println( )

( ) ,( )

expr "c fuera del if =" expr c

instrucciones( )

imprimir println( )

( ) ,( )

expr "d fuera del if =" expr d

instrucciones( )

sIf if( ) ( ) else( )

expr( ) &&( )

expr( ) ==( )

expr c expr 50

expr( ) ==( )

expr d expr 30

block { }

( ) ( )

instrucciones( )

varAsign puntosEntornos =( )

expr( ) +( )

expr puntosEntornos expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Uso de variable en entorno superior: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Uso de variable en entorno superior: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== If / Else ===="

instrucciones( )

varDcl mut puntosIfElse( ) ( ) ( )

type int assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "If simple"

instrucciones( )

sIf if( ) ( )

expr true block { }

( ) ( )

instrucciones( )

imprimir println( )

( )

expr "Condición verdadera"

instrucciones( )

varAsign puntosIfElse =( )

expr( ) +( )

expr puntosIfElse expr 1

instrucciones( )

imprimir println( )

( )

expr "\nIf-Else"

instrucciones( )

sIf if( ) ( ) else( )

expr true block { }

( )

instrucciones( )

imprimir println( )

( )

expr "Condición verdadera en if-else"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "Condición falsa en if-else"

instrucciones( )

sIf if( ) ( ) else( )

expr false block { }

( )

instrucciones( )

imprimir println( )

( )

expr "Esto no debería imprimirse"

block { }

( ) ( )

instrucciones( )

imprimir println( )

( )

expr "Condición falsa, ejecutando else"

instrucciones( )

varAsign puntosIfElse =( )

expr( ) +( )

expr puntosIfElse expr 1

instrucciones( )

imprimir println( )

( )

expr "\nIf-ElseIf-Else"

instrucciones( )

sIf if( ) ( ) else( )

expr true block { }

( )

instrucciones( )

imprimir println( )

( )

expr "Primera condición verdadera"

sIf if( ) ( ) else( )

expr true block { }

( )

instrucciones( )

imprimir println( )

( )

expr "Segunda condición verdadera, pero no se ejecuta"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "Ninguna condición verdadera"

instrucciones( )

sIf if( ) ( ) else( )

expr false block { }

( )

instrucciones( )

imprimir println( )

( )

expr "Primera condición falsa"

sIf if( ) ( ) else( )

expr true block { }

( ) ( )

instrucciones( )

imprimir println( )

( )

expr "Segunda condición verdadera"

instrucciones( )

varAsign puntosIfElse =( )

expr( ) +( )

expr puntosIfElse expr 1

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "Ninguna condición verdadera"

instrucciones( )

imprimir println( )

( )

expr "\n==== For Tipo While ===="

instrucciones( )

varDcl mut puntosForWhile( ) ( ) ( )

type int assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "For como while simple"

instrucciones( )

varDcl mut i :=( )

expr 0

instrucciones( )

varDcl mut suma :=( )

expr 0

instrucciones( )

sFor for( ) ( )

expr( ) <( )

expr i expr 5

block { }

( ) ( ) ( )

instrucciones( )

imprimir println( )

( ) ,( )

expr "i =" expr i

instrucciones( )

varAsign suma =( )

expr( ) +( )

expr suma expr i

instrucciones( )

varAsign i =( )

expr( ) +( )

expr i expr 1

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr suma expr 10

block { }

( ) ( )

instrucciones( )

varAsign puntosForWhile =( )

expr( ) +( )

expr puntosForWhile expr 1

instrucciones( )

imprimir println( )

( )

expr "OK For como while simple: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X For como while simple: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nFor como while anidado (patrón X)"

instrucciones( )

imprimir println( )

( )

expr "###Validacion Manual"

instrucciones( )

varDcl mut n( ) ( ) ( )

type int assign = expr 5

instrucciones( )

varDcl mut x( ) ( ) ( )

type int assign = expr 0

instrucciones( )

sFor for( ) ( )

expr( ) <( )

expr x expr n

block { }

( ) ( ) ( ) ( ) ( )

instrucciones( )

varDcl mut j :=( )

expr 0

instrucciones( )

varDcl mut fila :=( )

expr ""

instrucciones( )

sFor for( ) ( )

expr( ) <( )

expr j expr n

block { }

( ) ( )

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ||( )

expr( ) ==( )

expr x expr j

expr( ) ==( )

expr( ) +( )

expr x expr j

expr( ) -( )

expr n expr 1

block { }

( )

instrucciones( )

varAsign fila =( )

expr( ) +( )

expr fila expr "*"

block { }

( )

instrucciones( )

varAsign fila =( )

expr( ) +( )

expr fila expr " "

instrucciones( )

varAsign j =( )

expr( ) +( )

expr j expr 1

instrucciones( )

imprimir println( )

( )

expr fila

instrucciones( )

varAsign x =( )

expr( ) +( )

expr x expr 1

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr x expr 5

block { }

( ) ( )

instrucciones( )

varAsign puntosForWhile =( )

expr( ) +( )

expr puntosForWhile expr 1

instrucciones( )

imprimir println( )

( )

expr "OK For como while anidado: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X For como while anidado: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== For Clásico ===="

instrucciones( )

varDcl mut puntosForClasico :=( )

expr 0

instrucciones( )

imprimir println( )

( )

expr "For clásico simple"

instrucciones( )

varAsign suma =( )

expr 0

instrucciones( )

sFor for( ) ;( ) ;( ) ( )

varDcl mut i :=( )

expr 0

expr( ) <( )

expr i expr 5

varAsign i ++ block { }

( ) ( )

instrucciones( )

imprimir println( )

( ) ,( )

expr "i =" expr i

instrucciones( )

varAsign suma =( )

expr( ) +( )

expr suma expr i

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr suma expr 5

block { }

( ) ( )

instrucciones( )

varAsign puntosForClasico =( )

expr( ) +( )

expr puntosForClasico expr 1

instrucciones( )

imprimir println( )

( )

expr "OK For clásico simple: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X For clásico simple: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nFor clásico anidado (tabla de multiplicar)"

instrucciones( )

imprimir println( )

( )

expr "###Validacion Manual"

instrucciones( )

sFor for( ) ;( ) ;( ) ( )

varDcl mut i :=( )

expr 1

expr( ) <=( )

expr i expr 3

varAsign i ++ block { }

( ) ( )

instrucciones( )

sFor for( ) ;( ) ;( ) ( )

varDcl mut j :=( )

expr 1

expr( ) <=( )

expr j expr 3

varAsign j ++ block { }

( )

instrucciones( )

imprimir println( )

( ) ,( ) ,( ) ,( ) ,( )

expr i expr "x" expr j expr "=" expr( ) *( )

expr i expr j

instrucciones( )

imprimir println(

)

instrucciones( )

varAsign puntosForClasico =( )

expr( ) +( )

expr puntosForClasico expr 2

instrucciones( )

imprimir println( )

( )

expr "\n==== For Range ===="

instrucciones( )

varDcl mut puntosForRange( ) ( ) ( )

type int assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "For range con slice"

instrucciones( )

varDclSlice numeros( ) ( ) ( ) { }

assign = nuevoSlice [] type int ( )

contenidoSlice( ) ,( ) ,( ) ,( ) ,( )

expr 10 expr 20 expr 30 expr 40 expr 50

instrucciones( )

varAsign suma =( )

expr 0

instrucciones( )

varAsign sumaIndices =( )

expr 0

instrucciones( )

sFor for i , valor =

instrucciones range instrucciones( ) ( ) ( ) ( )

varDclStruct numeros {

)

instrucciones( )

imprimir println( )

( ) ,( ) ,( ) ,( )

expr "Índice" expr i expr "=" expr valor

instrucciones( )

varAsign suma =( )

expr( ) +( )

expr suma expr valor

instrucciones( )

varAsign sumaIndices =( )

expr( ) +( )

expr sumaIndices expr i

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr suma expr 150

block { }

( ) ( )

instrucciones( )

varAsign puntosForRange =( )

expr( ) +( )

expr puntosForRange expr 2

instrucciones( )

imprimir println( )

( )

expr "OK For range con slice: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X For range con slice: incorrecto"

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr sumaIndices expr 10

block { }

( ) ( )

instrucciones( )

varAsign puntosForRange =( )

expr( ) +( )

expr puntosForRange expr 1

instrucciones( )

imprimir println( )

( )

expr "OK For range con índices: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X For range con índices: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\n==== Switch/Case ===="

instrucciones( )

varDcl mut puntosSwitch( ) ( ) ( )

type int assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "Switch simple"

instrucciones( )

varAsign dia =( )

expr 1

instrucciones( )

sSwitch switch( ) { }

expr dia ( )

cases case( ) :( ) ( ) ( )

expr 1 instrucciones( )

imprimir println( )

( )

expr "Lunes"

instrucciones( )

varAsign puntosSwitch =( )

expr( ) +( )

expr puntosSwitch expr 1

cases case( ) :( ) ( )

expr 2 instrucciones( )

imprimir println( )

( )

expr "Martes"

cases case( ) :( ) ( )

expr 3 instrucciones( )

imprimir println( )

( )

expr "Miércoles"

cases case( ) :( ) ( )

expr 4 instrucciones( )

imprimir println( )

( )

expr "Jueves"

cases case( ) :( ) ( )

expr 5 instrucciones( )

imprimir println( )

( )

expr "Viernes"

cases case( ) :( ) ( )

expr 6 instrucciones( )

imprimir println( )

( )

expr "Sábado"

cases case( ) :( ) ( )

expr 7 instrucciones( )

imprimir println( )

( )

expr "Domingo"

cases default:( )

instrucciones( )

imprimir println( )

( )

expr "Día inválido"

instrucciones( )

imprimir println( )

( )

expr "\nSwitch con default"

instrucciones( )

varDcl mut numero( ) ( ) ( )

type int assign = expr 100

instrucciones( )

sSwitch switch( ) { }

expr numero ( )

cases case( ) :( ) ( )

expr 1 instrucciones( )

imprimir println( )

( )

expr "No se debería imprimir"

cases case( ) :( ) ( )

expr 2 instrucciones( )

imprimir println( )

( )

expr "No se debería imprimir"

cases default:( ) ( )

instrucciones( )

imprimir println( )

( )

expr "Número no reconocido, se ejecuta default"

instrucciones( )

varAsign puntosSwitch =( )

expr( ) +( )

expr puntosSwitch expr 1

instrucciones( )

imprimir println( )

( )

expr "\n==== Switch con break explícito ===="

instrucciones( )

varDcl mut numeroBreak( ) ( ) ( )

type int assign = expr 2

instrucciones( )

sSwitch switch( ) { }

expr numeroBreak ( )

cases case( ) :( ) ( )

expr 1 instrucciones( )

imprimir println( )

( )

expr "No se debería imprimir"

cases case( ) :( ) ( ) ( ) ( ) ( ) ( )

expr 2 instrucciones( )

imprimir println( )

( )

expr "Caso 2 - Se ejecuta este y debe detenerse"

instrucciones( )

varAsign puntosSwitch =( )

expr( ) +( )

expr puntosSwitch expr 1

instrucciones( )

break break

instrucciones( )

imprimir println( )

( )

expr "No debería ejecutarse si el break funciona"

instrucciones( )

varAsign puntosSwitch =( )

expr( ) -( )

expr puntosSwitch expr 1

cases case( ) :( )

expr 3 instrucciones( )

imprimir println( )

( )

expr "No se debería imprimir"

instrucciones( )

imprimir println( )

( )

expr "\n==== Break ===="

instrucciones( )

varDcl mut puntosBreak( ) ( ) ( )

type int assign = expr 0

instrucciones( )

imprimir println( )

( )

expr "Break en for infinito"

instrucciones( )

varDcl mut contador( ) ( ) ( )

type int assign = expr 0

instrucciones( )

varDcl mut suma( ) ( ) ( )

type int assign = expr 0

instrucciones( )

sFor for( ) ( )

expr true block { }

( ) ( ) ( ) ( )

instrucciones( )

imprimir println( )

( ) ,( )

expr "contador =" expr contador

instrucciones( )

varAsign suma =( )

expr( ) +( )

expr suma expr contador

instrucciones( )

varAsign contador =( )

expr( ) +( )

expr contador expr 1

instrucciones( )

sIf if( ) ( )

expr( ) >=( )

expr contador expr 5

block { }

( )

instrucciones( )

break break

instrucciones( )

sIf if( ) ( ) else( )

expr( ) ==( )

expr suma expr 10

block { }

( ) ( )

instrucciones( )

varAsign puntosBreak =( )

expr( ) +( )

expr puntosBreak expr 1

instrucciones( )

imprimir println( )

( )

expr "OK Break en for infinito: correcto"

block { }

( )

instrucciones( )

imprimir println( )

( )

expr "X Break en for infinito: incorrecto"

instrucciones( )

imprimir println( )

( )

expr "\nBreak en for clásico"

instrucciones( )

varAsign suma =( )

expr 0

instrucciones sFor instrucciones( )

sFor for

instrucciones( )

varAsign i =( )

expr 0